Ex No: 2c

IMPLEMENTATION OF BACKWARD CHAINING

Aim*:*

To implement backward chaining.

Procedure:

Program:

```
# Facts: The knowledge base containing information about facts
facts = {
    'a': True,
    'b': True,
    'c': False
}
# Rules: List of rules where each rule is a tuple (conclusion, conditions)
rules = [
    ('d', ['a', 'b']),  # d can be concluded if a and b are true
    ('e', ['b', 'c']),  # e can be concluded if b and c are true
    ('f', ['d', 'e'])  # f can be concluded if d and e are true
]
# Backward Chaining Function
def backward_chaining(goal, facts, rules):
    # Check if the goal is already a fact
    if goal in facts:
        return facts[goal]
```

```
        # Check if the goal can be concluded from any rule
    for rule in rules:
        head, body = rule
        if head == goal:
            # Check if all the conditions in the body of the rule are true
            if all(backward_chaining(fact, facts, rules) for fact in body):
                return True

    # If the goal cannot be proved
    return False

# Example Usage:
goal = 'f'

if backward_chaining(goal, facts, rules):
    print(f"The goal '{goal}' can be achieved.")
else:
    print(f"The goal '{goal}' cannot be achieved.")
```

Output:

Expected Output for the Goal `'f'`:
The goal 'f' cannot be achieved.

Expected Output for the Goal `'d'`:
The goal 'd' can be achieved.

Terminal - Ex 2c: Implementation of Backward Chaining

```
$ py code/ex2c.py

Expected Output for the Goal 'f':
The goal 'f' cannot be achieved.

Expected Output for the Goal 'd':
The goal 'd' can be achieved.
```